

# Towards Verifiable FHE in Practice: Proving Correct Execution of TFHE’s Bootstrapping using plonky2

Louis Tremblay Thibault  
louis.tremblay.thibault@zama.ai  
Zama  
Paris, France

Michael Walter  
michael.walter@zama.ai  
Zama  
Paris, France

## Abstract

In this work we demonstrate for the first time that a full FHE bootstrapping operation can be proven using a SNARK in practice. We do so by designing an arithmetic circuit for the bootstrapping operation and prove it using plonky2. We are able to prove the circuit on an AWS Hpc7a instance in under 20 minutes. Proof size is about 200 kB and verification takes less than 10 ms. As the basis of our bootstrapping operation we use TFHE’s programmable bootstrapping and modify it in a few places to more efficiently represent it as an arithmetic circuit (while maintaining full functionality and security). In order to achieve our results in a memory-efficient way, we take advantage of the structure of the computation and plonky2’s ability to efficiently prove its own verification circuit to implement a recursion-based IVC scheme. Lastly, we present a security proof in the UC model that captures active attacks in real world applications of verifiable FHE and augment our prototype to fit such applications.

## 1 Introduction

There are two emerging cryptographic technologies with a host of applications in practice: Fully Homomorphic Encryption (FHE) and Succinct Non-interactive Arguments of Knowledge (SNARKs). FHE allows arbitrary computation on encrypted data, while SNARKs enable proving the correct execution of arbitrary computation with short proofs and verification time sublinear in the size of the computation. It is not hard to see the vast number of possible use cases for each of these technologies in practice, but in this work we are interested in the combination of the two. Specifically, we investigate to what extent it is possible to prove the correct execution of FHE operations using SNARKs in practice.

Combining FHE with SNARKs is enormously appealing as this has the potential to entirely replace solutions to secure outsourcing of computing based on hardware modules, which are riddled with practical attacks and ultimately only achieve a shift of trust to the hardware vendor. In contrast, a verifiable FHE scheme would allow outsourcing computation and reduce trust to cryptographic, i.e., mathematical assumptions using minimal interaction. Furthermore, such a scheme would thwart CCA-style attacks, to which FHE schemes are known to be inherently vulnerable [23], which means that in a practical deployment, one needs to be very prudent in its use of FHE in order not to fall victim to attacks outside of the security model.

Unfortunately, despite a large amount of research and significant progress over the past one and a half decades, FHE operations still incur a significant overhead over their cleartext counterparts. Even worse, any truly *fully* homomorphic scheme we know of to date relies on a bootstrapping operation to reduce noise in ciphertexts,

which accumulates during homomorphic operations and may lead to incorrect decryption if not handled correctly. In all current FHE schemes, this bootstrapping is the costliest operation.

On the other hand, SNARKs themselves incur a significant overhead over the computation to prove, with many practical SNARKs having proving complexity superlinear in the size of the computation.<sup>1</sup> Furthermore, since the proof generation typically requires to keep the entire trace in memory, the memory requirement of SNARKs grows at least linearly in the computation length, which renders the memory complexity the bottleneck for long computations. There are techniques to mitigate this issue for *structured* computations. We will come back to this later.

So it is not surprising that the bootstrapping operation represents a formidable challenge for SNARKs. While some works have considered proving *levelled* homomorphic operations [29, 47], as far as we are aware, there were no published attempts of applying a SNARK to an FHE bootstrapping operation, let alone successful attempts, prior to our work.<sup>2</sup> In this work we seek to remedy this state of affairs and demonstrate the practicality of a fully verifiable bootstrapping.

Finally, we note that the security of FHE schemes is usually only considered under passive attacks (IND-CPA), while classic PKE schemes are typically expected to be secure also under active attacks (IND-CCA). Unfortunately, capturing active attacks using classic game-based indistinguishability notions is notoriously hard for FHE due to the inherent malleability of ciphertexts. This leads to weak or arguably over-complex definitions [1, 15, 38, 47, 48]. We address this issue by modeling a typical use case of verifiable FHE as an MPC application and prove its security in the UC model.

### 1.1 Contribution

In this work we use an argument system for general purpose computation, plonky2 [39], in order to allow the evaluator of a TFHE bootstrapping [22] to prove that it did so correctly. Note that the proof is publicly verifiable, not just by the party holding the decryption key (or some other kind of secret verification key). In order to use plonky2, we need to rewrite the bootstrapping algorithm in terms of an arithmetic circuit over a finite field. In this work we present a number of tweaks to TFHE to reduce its circuit size. Still, the main challenge is the sheer size of the bootstrapping circuit, which is too large to be handled in practice. To address this, we show how to exploit incrementally verified computation (IVC) [46] to take advantage of its inherent structure and provide an experimental evaluation.

<sup>1</sup>With *size* we mean here the size of the circuit used to perform the computation in the arithmetic circuit model.

<sup>2</sup>A subsequent work [37] does consider the same problem. See below for a comparison.

We compare our experimental results to using general-purpose zero-knowledge virtual machines (zkVMs) to prove correct execution of the programmable bootstrapping (PBS) [14, 42]. Such zkVMs promise to be easy to use at the cost of introducing an overhead. To evaluate the extent of the overhead compared to our specially crafted circuit, we use a straight-forward implementation of the PBS and apply the zkVMs using a variety of test machines. The first observation is that neither of the two zkVMs we tested were actually able to prove an entire PBS due to performance limitations even on powerful machines (and in one case even a computer cluster), while this was no problem at all for our plonky2-based implementation even on moderate machines. It is plausible that tweaking the implementation on the zkVMs could solve the issue, but this was out of scope of this project. To still obtain a quantitative comparison, we performed micro-benchmarks. The results indicate that our implementation outperforms the zkVMs by at least two orders of magnitude.<sup>3</sup>

To the best of our knowledge, our results demonstrate for the first time that generating a proof for a bootstrapping operation is practically feasible: we are able to prove correctness of a TFHE-like bootstrapping with secure parameters in under 20 minutes on an AWS Hpc7a instance. While this is still likely to be too costly for many applications, others might already be able to take advantage of a fully verifiable FHE scheme. As an example, consider a blockchain protocol that allows smart contracts on encrypted data [25, 43]. Here, verifiable FHE operations have the potential to replace consensus protocols and thus reduce the computational load of validators. The given proving time could be acceptable in this setting, if this is deployed akin to hybrid rollups, where a proof is only required in case of a dispute.

Furthermore, we argue for a security model that formalizes the settings in which our verifiable FHE construction can be used securely. In more detail, we note that adding a proof of correct evaluation to the ciphertext allows to strengthen the security model to some forms of active attacks, but it is not obvious what the precise security model is. A sound definition of a security notion achieved by this construction was recently provided [48], but it suffers from similar issues as previous work in being overly complex. We observe that the key issue in game-based security definitions for FHE meant to capture active security is that FHE ciphertexts are inherently malleable. The difficulty is to cleanly separate “benign” modifications of the ciphertext from malicious ones. To address this issue, we diverge from the game-based definitional framework and turn to simulation-based definitions. Specifically, inspired by [40], we model typical secure outsourcing of computation as an MPC application and give a proof in the UC model. The advantage of the simulation-based approach is that we do not need to explicitly define malicious and benign modifications of the ciphertext, since the definition simply states that the adversary learns everything the ideal functionality leaks about the plaintexts, but nothing beyond that.

Lastly, by slightly augmenting our implementation, we show that we are able to prove the correct evaluation of simple functions over

ciphertexts (e.g., a weighted sum followed by a unitary function) at a very low cost, potentially unlocking applications in the field of privacy-preserving machine learning (ML) that fit within our MPC model.

## 1.2 Choice of FHE scheme and SNARK

*FHE Scheme.* Since our goal is fairly ambitious, we try to make our lives as easy as possible. In particular, we choose as our target the FHE scheme with the lightest known bootstrapping operation, namely TFHE [8, 22, 24]. We take the liberty to modify TFHE at a few places to make it more amendable to our target SNARK. These modifications maintain the functionality of the bootstrap, but might make it slightly less efficient. If the modifications yield a faster proof generation, this is likely a worthwhile trade-off depending on the overall system. The most significant modification we apply is to use a SNARK-friendly prime modulus  $q \approx 2^{64}$  instead of a power of 2, because most efficient SNARKs only natively support arithmetic circuits over finite fields. This way we avoid emulating the arithmetic in the ring  $\mathbb{Z}_{2^{64}}$  within the SNARK field. While there are attempts to construct SNARKs for ring arithmetic [29], this comes with its own caveats, like designated verifier and relatively poor performance.

*SNARK.* There are a number of SNARK implementations for general purpose computation available and we selected the SNARK for our work based on the following criteria. In order to enable as many applications as possible, we target a transparent, publicly verifiable SNARK with sublinear verifier. For efficiency reasons, we require native support for arithmetic in fields of size  $\approx 2^{64}$  and, ideally, support for efficiency improvements for structured computation like loops, since the core of TFHE’s bootstrapping is essentially a large loop.

With these criteria in mind, plonky2 provides a suitable candidate. It relies on the PLONK arithmetization [28] in combination with a polynomial commitment scheme based on hash functions, namely FRI [6]. It uses as a base field  $\mathbb{F}_p$  with  $p = 2^{64} - 2^{32} + 1$ , which meets our requirement on the modulus, and, as an added bonus, is plausibly post-quantum secure, which is also true of TFHE. plonky2 is optimized for recursion, which allows us to construct *incrementally verifiable computation* (IVC) [46], a technique to prove the correct execution of loops more efficiently than simply rolling them out in a circuit, which will come in handy.

## 1.3 Related Work

There is a line of research considering verifiable computation on encrypted data. The first results in this area [31, 33] are mainly of theoretical interest as they rely on heavy machinery like combining garbled circuits with FHE or functional encryption. The study of systems combining mechanisms for verifiable computation with FHE, as we do in this work, was initiated in [26] and continued in [10, 27, 29, 47]. While these works promise better concrete efficiency than the aforementioned schemes, they still seem to be impractical and sidestep the complexity of bootstrapping by restricting to levelled HE schemes.

A few works have started investigating an approach based on performing the integrity check in the plaintext space [3, 18, 19, 30], with [3] and [18, 19] claiming practical efficiency. However, this

<sup>3</sup>We remark that some zkVMs like [42] have some advanced features, like precompiled circuits, that we did not explore in this project. We believe it is an interesting open question if more advanced usage of the zkVMs could yield results comparable to ours in a simpler way.

approach has the significant drawback that it requires decryption in order to verify the computation. This has two highly undesirable consequences: First, only the party with the secret key can verify the computation. While this might be acceptable in some applications, it does rule out many others. And second, it leaves the FHE scheme vulnerable to active attacks. As we will show in Section 6, performing integrity checks on the ciphertext allows to strengthen the security model significantly. Unfortunately, this is not the case if the client needs to decrypt the ciphertext before being able to verify. It can be verified that our proof of security does not hold for such constructions and indeed, it is not too hard to see that they are vulnerable to the attacks presented in [23].

Recent work [15, 38] has successfully managed to meaningfully define security of FHE under active attacks using game-based indistinguishability notions. However, these definitions do not cover the construction in this work and the corresponding constructions are inefficient in practice, since they require to include validity checks over ciphertexts into the proof of correct evaluation and make very strong assumptions about the employed SNARK. For example, these constructions cannot be instantiated using plonky2, since they require blackbox and straight-line extractability.

Finally, HasteBoots [37] appears after this work and claims to allow proofs of correct bootstrapping evaluation to be generated in the order of seconds. It differs from this work in two key aspects. First of all, looking ahead to Section 5, the prover time from [37] is better than ours (4 s instead of 1080 s). However, the proof size and verifier time from [37] are orders of magnitude worse: verifier times jump from less than 10 ms to more than 300 ms and proof size goes from under 200 kB to above 150 MB. As such, HasteBoots can be seen as a follow-up work that trades prover time for proof size and verifier time. Furthermore, at the time of writing, the authors have not provided an implementation that can be used to replicate their results.

*Future Work.* As hinted at above, our work makes use of recursion-based IVC. There is a recent line of work constructing more efficient IVC from folding schemes [13, 16, 35]. We chose to focus on recursion, because folding schemes require the commitment scheme of the SNARK to be homomorphic. However, until very recently, there was no homomorphic scheme suitable for our application, due to large field size and/or trusted setup and/or inefficient verifier. This changed very recently with [11] and an exciting open question is if more efficient provers can be obtained using this new lattice-based folding.

*Outline.* We begin with some background on LWE and TFHE in Section 2, which should be sufficient to follow the rest of the paper except for the security proof of Section 6. For the sake of readability, we introduce the required definitions for the security proof separately in Section 6. Section 3 describes our implementation of the core functionality of the bootstrapping (the blind rotation) and in Section 4 we show how to extend it to the full bootstrapping. In Section 5 we present experimental results and discuss applications in Section 7.

## 2 Preliminaries

*Notation.* Throughout we will use the parameters  $N, q \in \mathbb{Z}$ , where  $N$  is a power of 2. If  $N$  is clear from context, we let  $R = \mathbb{Z}[X]/(X^N + 1)$  and  $R_q = R/qR$ . Note that  $R = \mathbb{Z}$  and  $R_q = \mathbb{Z}_q$  when  $N = 1$ . Elements in  $R_q$  (for any  $N$ ) are denoted by lower case letters, vectors over  $R_q$  by bold lower case letters and for a vector  $\mathbf{a} \in R_q^k$  we denote by  $a_i$  its  $i$ -th component. Similarly, if  $a \in R_q$  we refer to  $a_i \in \mathbb{Z}_q$  as its  $i$ -th coefficient, i.e. we have  $a = \sum_i a_i X^i$ . For an element  $a \in R$  we consider its norm  $|a|$  to be the  $\infty$ -norm of its coefficient vector and we extend the norm to elements of  $R_q$  by lifting them to  $R$ , picking the representative with coefficients between  $-q/2$  and  $q/2$ .

### 2.1 (G)LWE

*Definition 2.1.* Let  $N, q, k \in \mathbb{Z}$  with  $N$  a power of 2. Let  $R = \mathbb{Z}[X]/(X^N + 1)$  and  $R_q = R/qR$ . Finally, let  $\mathcal{X}$  be a “small” distribution over  $R_q$ . Then, for a fixed  $\mathbf{s} \in R_q^k$  the *General LWE distribution*  $\text{GLWE}_{N,q,k,\mathcal{X}}^{\mathbf{s}}$  is defined as  $(\mathbf{a}, b = \langle \mathbf{a}, \mathbf{s} \rangle + e)$  where  $\mathbf{a}$  is chosen uniformly at random from  $R_q^k$  and  $e$  is chosen from  $\mathcal{X}$ .

Let  $\mathcal{S}$  be some distribution over  $R_q^k$ . The *GLWE problem*  $\text{GLWE}_{N,q,k,\mathcal{X}}^{\mathcal{S}}$  is to distinguish the distribution  $\text{GLWE}_{N,q,k,\mathcal{X}}^{\mathbf{s}}$  from the uniform distribution over  $R_q^{k+1}$ , where  $\mathbf{s} \leftarrow \mathcal{S}$ .

The  $\text{LWE}_{q,k,\mathcal{X}}^{\mathcal{S}}$  problem is a special case of the  $\text{GLWE}_{N,q,k,\mathcal{X}}^{\mathcal{S}}$  where  $N = 1$ . TFHE assumes a secret distribution  $\mathcal{S}$  that is uniform over elements in  $R_q^k$  with binary coefficients and thus we will assume this secret distribution throughout. With suitable choice for the error distribution  $\mathcal{X}$  (e.g. discrete or rounded Gaussian with sufficiently large variance) and ring dimension  $k$  the corresponding  $\text{GLWE}_{N,q,k,\mathcal{X}}^{\mathcal{S}}$  problem is considered to be hard. It is standard practice to estimate the concrete security of a specific LWE instance using the lattice estimator [2].

### 2.2 TFHE

TFHE is a secret key FHE scheme based on (G)LWE. In the following we try to give a succinct intuitive description of TFHE that we hope is detailed enough to follow the rest of the work without cluttering it with too much formal notation. For a more rigorous description, we refer to [22] and follow up work, or the survey [34].

The basic ciphertexts in TFHE are simple LWE ciphertexts, but internally it uses a range of other ciphertexts based on GLWE. Since we need to represent the entire bootstrapping as a circuit, we make use of all types of ciphertexts and thus we introduce them next.

#### 2.2.1 Ciphertext Types.

*(G)LWE ciphertext.* Let  $N, q, k \in \mathbb{Z}$ ,  $\mathcal{X}$  be GLWE (or simply LWE in case  $N = 1$ ) parameters. For a message  $m \in R_p$ , we define its (G)LWE encryption to be  $(\mathbf{a}, b = \langle \mathbf{a}, \mathbf{s} \rangle + e + \mu)$ , where  $\mathbf{a} \in R_q^k$  is uniformly random,  $\mathbf{s} \in R_q^k$  is chosen from the uniform binary distribution and  $e$  from  $\mathcal{X}$ . By the hardness of (G)LWE this is a semantically secure ciphertext. It can be decrypted using  $\mathbf{s}$  if  $\mu$  represents a suitable encoding of a message that is robust w.r.t. the error distribution. For example, let  $p < q \in \mathbb{Z}$  be a plaintext modulus and define  $\Delta = \lfloor q/p \rfloor$ . For a message  $m \in R_p$ , we may define its encoding  $\mu$  as  $\Delta \cdot m$ , which allows to recover  $m \in \mathbb{Z}_p$  by rounding. In

the context of LWE ciphertexts we typically denote the dimension by  $n$  instead of  $k$ . Note that (G)LWE ciphertexts are additively homomorphic and may be multiplied with “small” elements in  $R_q$ , where smallness is determined such that the resulting ciphertext can still be correctly decrypted given the error distribution and the encoding.

*GLew Ciphertext.* GLew ciphertexts (where the “Lev” stands for *levelled*) are a way to extend (G)LWE ciphertexts in order to allow for multiplication with arbitrary constants. It is based on the standard approach of decomposition: for an element  $a \in R_q$  and parameters  $B$  and  $\ell$ , denote by  $\text{Dec}_{B,\ell}(a) \mapsto \mathbf{a}$  the transformation such that  $\mathbf{a} \in R_q^\ell$ ,  $|a_i| \leq B/2$  and  $\sum_{i=1}^\ell \lfloor \frac{q}{B^\ell} \rfloor a_i \approx a$ . With this decomposition at hand, we define the GLew encryption of  $m \in R_q$  with parameters  $B$  and  $\ell$  to be the set of (G)LWE encryptions of  $(\lfloor \frac{q}{B^\ell} \rfloor \cdot m$  for all  $i \in \{1 \dots \ell\}$ . Note that such a ciphertext can be multiplied with an arbitrary element  $a \in R_q$  by first decomposing  $a$  using parameters  $B$  and  $\ell$  and taking the inner product with the GLew ciphertext. Since all components of  $\text{Dec}_{B,\ell}(a)$  are small the result is an (G)LWE encryption of  $a \cdot m$  by the homomorphic properties of the (G)LWE ciphertexts (and assuming suitable parameters).

*GGSW Ciphertext.* While GLew ciphertexts allow to multiply encrypted values with arbitrary constants, we would also like to be able to efficiently multiply encrypted values with each other. This can be achieved using GGSW ciphertexts (named after [32]). The idea is to encrypt  $m$  as a GLew ciphertext and for each element  $s_i$  of the secret key  $\mathbf{s} \in R_q^k$ , additionally encrypt  $m \cdot s_i$  as a GLew ciphertext. This set of  $k + 1$  GLew ciphertexts forms the GGSW ciphertext. By the properties of GLew ciphertexts, this allows to perform a multiplication while at the same time homomorphically decrypting a ciphertext  $(\mathbf{a}, b = \langle \mathbf{a}, \mathbf{s} \rangle + m' + e)$ . This is done by homomorphically computing  $b \cdot m$  and  $a_i \cdot s_i \cdot m$  and using the additive homomorphism of GLWE ciphertexts. Note that  $m$  should not be too large as this would blow up the error. In TFHE, the message  $m$  is usually a key bit and thus binary, so clearly small. In summary, a GGSW ciphertext allows us to multiply a GLWE ciphertext with a GLew ciphertext and to obtain a GLWE ciphertext encrypting the product of the two plaintexts (as long as the plaintext in the GGSW ciphertext is sufficiently small). This operation is typically called the *external product*.

**2.2.2 Programmable Bootstrapping.** The PBS of TFHE receives as input

- the LWE ciphertext  $(\mathbf{a}, b = \langle \mathbf{a}, \mathbf{s} \rangle + e + \Delta \cdot m) \in \mathbb{Z}_q^n$  to bootstrap, where the corresponding secret key is  $\mathbf{s} \in \{0, 1\}^n$ ,
- an element  $t \in R_q$  that allows to encode a function<sup>4</sup>  $f : \mathbb{Z}_p \mapsto \mathbb{Z}_p$  into the bootstrap,
- the bootstrapping key (*bsk*) as a collection of GGSW ciphertexts encrypting the individual bits  $s_i \in \{0, 1\}$  of the secret key under a bootstrapping secret key  $\mathbf{s}' \in R_q^k$  with binary coefficients, and
- a key switching key (*ksk*) as a collection of GLew ciphertexts encrypting the coefficients of the bootstrapping key under the secret key  $\mathbf{s}$ .

<sup>4</sup>There is a requirement for the function to be negacyclic, but we omit details since it is irrelevant for our work.

It outputs a ciphertext  $(\mathbf{a}', b' = \langle \mathbf{a}', \mathbf{s} \rangle + e' + \Delta \cdot f(m))$ , where  $e'$  only depends on the *bsk* and *ksk*, not on  $e$ . For suitable parameters, we have that  $|e'| < |e|$ . Combining this with the additive homomorphism of LWE ciphertexts we obtain a Fully Homomorphic Encryption scheme.

The PBS consists of the following four steps. See Figure 1 for an illustration.

*Mod Switch.* We embed the input ciphertext into the group  $\langle X \rangle \subset R_q$ , which is of size  $2N$ . So in order to match up the moduli, we first perform a modulus switch. In particular, this takes as input the ciphertext  $(\mathbf{a}, b) \in \mathbb{Z}_q^{n+1}$  and outputs  $(\mathbf{a}', b') \in \mathbb{Z}_{2N}^{n+1}$ , where

$$a'_i = \left\lfloor \frac{a_i 2N}{q} \right\rfloor$$

and similar for  $b'$ .

*Blind Rotation.* The blind rotation is the core of the PBS. We begin its description by introducing a homomorphic ciphertext multiplexer (CMUX) operation: given two GLWE ciphertext  $\mathbf{c}_0, \mathbf{c}_1 \in R_q^{k+1}$  and a GGSW encryption  $C_\mu$  of a bit  $\mu \in \{0, 1\}$ , all under the same key  $\mathbf{s} \in R_q^k$ , we can compute the GLWE ciphertext

$$\mathbf{c} = (\mathbf{c}_1 - \mathbf{c}_0) \odot C_\mu + \mathbf{c}_0$$

where  $\odot$  corresponds to the external product described in Section 2.2.1. By the additive homomorphism and the properties of the external product,  $\mathbf{c}$  will encrypt the same plaintext as  $\mathbf{c}_\mu$ .

We are now ready to describe the blind rotation. Let  $(\mathbf{a}, b) \in \mathbb{Z}_{2N}^{n+1}$  be the ciphertext after the mod switch. The blind rotation begins by constructing a trivial GLWE ciphertext  $(\mathbf{0}, X^{-b} \cdot t)$ , where  $\mathbf{0} \in R_q^k$  is the all zero vector of size  $k$ . Then, it iterates over the elements  $a_i$  of  $\mathbf{a}$ , where the output GLWE ciphertext  $\mathbf{c}$  from the previous iteration is multiplied element-wise by  $X^{a_i}$ . The two ciphertexts  $\mathbf{c}$  and  $X^{a_i} \cdot \mathbf{c}$  are input to a homomorphic CMUX, with the control bit being the corresponding part of the *bsk* which is itself a GGSW ciphertext encrypting  $s_i$ . Accordingly, the result is a ciphertext encrypting the same plaintext as  $X^{a_i s_i} \cdot \mathbf{c}$ . After executing the full loop, the result is a GLWE ciphertext encrypting  $X^{-b + \sum_i a_i s_i} \cdot t = X^{-b + \langle \mathbf{a}, \mathbf{s} \rangle} \cdot t = X^{-m - e} \cdot t$ . Note that in  $R_q$ , this corresponds to a negacyclic rotation of  $t$  by  $m + e$  positions. By redundantly embedding the function  $f$  into the test polynomial  $t$ , we can ensure that the error  $e$  is rounded away and the resulting ciphertext contains an encryption of  $\Delta \cdot f(m)$  in its constant coefficient.

*Sample Extraction.* The goal of sample extraction is to convert a GLWE ciphertext into an LWE ciphertext encrypting the constant coefficient of the GLWE ciphertext, and where the key is a vector of bits corresponding to the concatenation of coefficient vectors in the GLWE secret key. We describe the special case of  $k = 1$ , since the generalization is straight-forward. So, given  $(\mathbf{a}, b) \in R_q^2$  we seek to construct  $(\mathbf{a}', b') \in \mathbb{Z}_q^{N+1}$  such that  $(b - \mathbf{a} \cdot \mathbf{s})_0 = b' - \langle \mathbf{a}', \mathbf{s}' \rangle$ , where  $\mathbf{s}'$  is as described above. We note that

$$\mathbf{a} \cdot \mathbf{s} = \sum_i a_i \cdot s_i X^i = \sum_i (X^i a_i) s_i$$

Since addition in  $R_q$  is elementwise, we may set  $a'_i = (X^i a_i)_0$  and  $b' = b_0$  in order to achieve our goal.

**Key Switch.** The key switch is a classic LWE type operation that follows from the observation that GLev ciphertexts can be used to homomorphically decrypt a GLWE ciphertext. Let  $(a, b) \in R_q^{k+1}$  be a GLWE ciphertext with corresponding secret key  $s \in R_q^k$ . We would like to obtain a ciphertext  $(a', b') \in R_q^{k'+1}$  encrypting the same message as  $(a, b)$  but under the secret key  $s' \in R_q^{k'}$ . We can do so by constructing a key switching key (*ksk*) that consists of GLev encryptions of  $s_i$  under  $s'$ . Then, using the fact that we can multiply these ciphertexts with arbitrary constants using decomposition, we can homomorphically compute a ciphertext encrypting  $b - \langle a, s \rangle$  under  $s'$ , which yields the desired ciphertext. In the context of TFHE this operation has classically been applied to the LWE ciphertexts resulting from the sample extraction, but we remark that it may also be applied to GLWE ciphertexts.

Finally, we describe a slight modification of a GGSW ciphertext that also allows to perform a key switch, as already noticed in [9]. Recall that a GGSW ciphertext consists of a set of GLev ciphertexts of messages  $m \cdot s_i$ , where the  $s_i$  are the elements of the secret key  $s$ . This allows to multiply a GLWE ciphertext and a GGSW ciphertext (both under secret key  $s$ ) to obtain a GLWE encryption of the product of the two messages under the same secret key  $s$ . Now assume that we have a GLWE ciphertext encrypted under  $s$  and construct a GGSW ciphertext using GLev encryptions of the elements  $m \cdot s_i$  but under a different key  $s'$ . We can still apply the external product to obtain a GLWE ciphertext of the product, but the resulting ciphertext will be an encryption under  $s'$ . In other words, by modifying the GGSW encryption and setting  $m = 1$ , we can also use the external product to perform a GLWE key switch. This will be useful in Section 4.2.

### 2.3 Parameters

As is plain from above description, there are a lot of parameters involved that impact the security, correctness and performance of TFHE. We do not go into details just yet but we remark that TFHE is typically instantiated with the ciphertext modulus  $q = 2^{64}$  (see e.g. [21, 50]). The other parameters are the result of a complex optimization procedure, but for concreteness the reader may consider Table 2.

## 3 Blind Rotation

Since the blind rotation is the core of the PBS, we begin by describing our circuit for this part of the PBS. We start out with a circuit for one iteration and then explain how we scale to a full blind rotation.

### 3.1 One Step of the Blind Rotation

One of the bottleneck operations during a step of the blind rotation is polynomial multiplication in  $R_q$ . Implementations like [50] or the one accompanying [21] use an FFT on floating point numbers, which are very inefficient to realize in the arithmetic circuit model. Luckily, the choice of modulus  $q = 2^{64} - 2^{32} + 1$  admits performing this multiplication using the NTT, so here we diverge from common implementations and use an NTT circuit instead.

The second main operation is multiplication by the monomial  $X^a$ , where  $a \in \mathbb{Z}_{2N}$  is an input. This corresponds to a negacyclic rotation by  $a$  in the ring  $R_q$ , which is a rather trivial (and linear)

operation on a CPU. However, in the circuit model it is not quite as easy, since  $a$  is not known during circuit construction and we cannot “rewire” a circuit during evaluation. Note that this operation would be trivial in the circuit model, if  $a$  was a fixed constant. So our solution to this problem is to implement subcircuits for negacyclic rotations by powers of two. Then we apply each of the subcircuits and each time select the rotated or not rotated polynomial using a MUX and the corresponding bit of the binary decomposition of  $a$  as control bit. See Figure 2 for an illustration. This is a circuit of size  $O(N \log N)$  and thus significantly more expensive than on a CPU. All other operations (addition, decomposition) are readily available in plonky2 and are easily generalized to polynomials.

### 3.2 Scaling to Full Blind Rotation

The obvious way to scaling the blind rotation step to  $n$  steps is to build a large circuit with  $n$  subcircuits performing one step each. While this works in theory, the circuit size blows up, since  $n$  is very large. In fact, in our experiments we were only able to do this for small  $n$ , cf. Section 5.2. For larger  $n$ , our test machines ran out of memory, which quickly becomes the bottleneck.

Another easy approach to scaling the blind rotation is to simply prove each step individually and send the proofs and intermediate results to the verifier. The verifier can check each of the proofs. This achieves a proving time that is linear in the number of steps and can be performed with memory equivalent to one step. The issue with this approach is the proof size and verifier complexity: the proof now consists of  $n$  smaller proofs and  $n$  GLWE ciphertexts (the intermediate results) and the verifier needs to check all individual proofs. In other words, the proof and verifier are not succinct as they are linear in the circuit size. With our example parameters (cf. Table 2), each ciphertext has size about 4 kB, so  $n > 2^9$  ciphertexts alone amount to about 2 MB, without even considering the substantially larger proofs.<sup>5</sup> In some applications this might be acceptable, but typically this is considered too large and the burden on the verifier too costly.

Clearly, we can use a hybrid strategy to reduce the proof size and verifier complexity. If we are working on a machine that is able to prove  $t$  steps of the blind rotation at a time, we may take advantage of this and cut the number of intermediate results and inner proofs down by a factor  $t$ .

**3.2.1 Blind Rotation Based on IVC.** As noted in Section 1, plonky2 supports recursion and thus allows constructing IVC. The general idea of IVC to prove the correct execution of a loop is the following. Let  $F$  be the function describing the step function of the loop, i.e., we want to prove  $y = F^n(x)$ , where  $F^n(x)$  corresponds to applying  $F$  successively  $n$  times to  $x$ . We may augment  $F$  to obtain a function  $F'$  that takes as input the (public) initial value  $x$ , a private prover input  $y_i$ , and, also as a private witness, a proof  $\pi_i$ .  $F'$  outputs  $y_{i+1}$  while also verifying that the proof  $\pi_i$  is valid with respect to  $F'$  itself (for input  $x$  and output  $y_i$ ). For an illustration of a circuit  $F'$  computing  $F'$  see Figure 3. A proof for this circuit attests to the correctness of the combined statement: 1)  $y_{i+1} = F(y_i)$  and 2) the verifier accepts  $\pi_i$  as a proof of  $y_i = F(x)$ . The prover may now

<sup>5</sup>We remark that it might be possible to compress the set of proofs into a single smaller proof using recursion, but this will certainly not work for the intermediate results, which need to be sent and checked in any case.

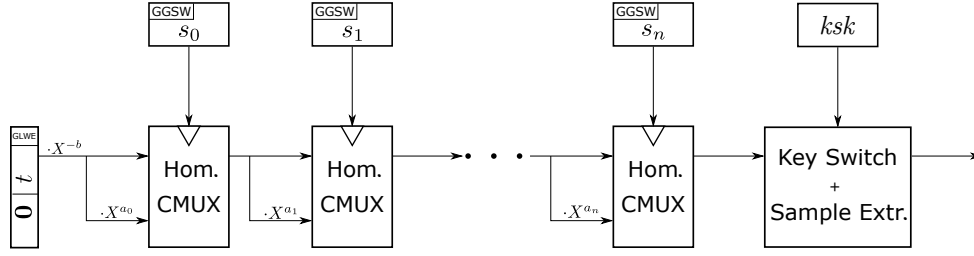
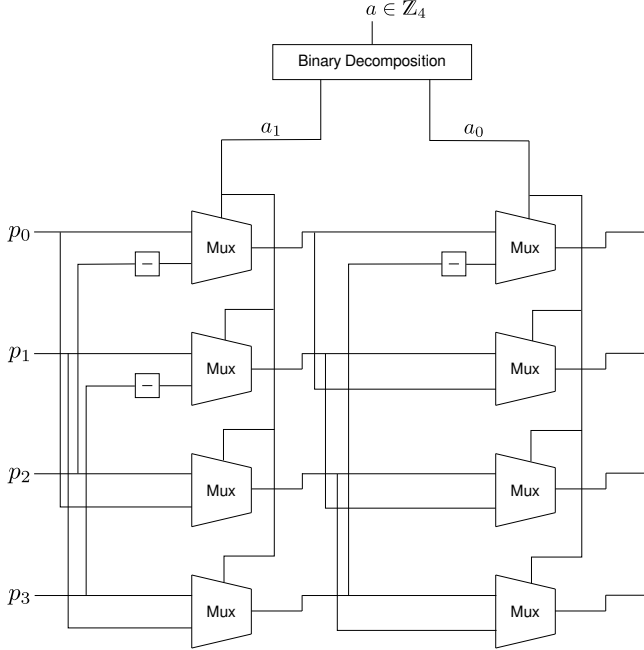
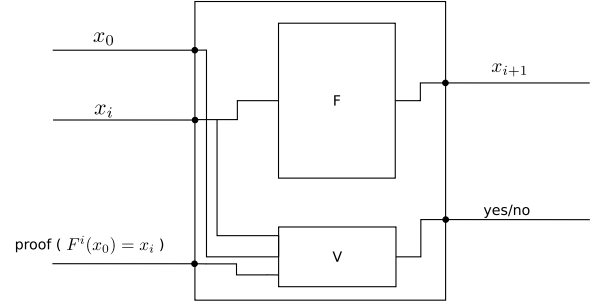


Figure 1: Illustration of TFHE's PBS (without mod switch)


 Figure 2: Circuit for multiplication of polynomial  $p = \sum_i p_i X^i \in \mathbb{Z}_q[X]/(X^4 + 1)$  by  $X^a$ 

successively prove  $y_i = F'(y_{i-1}, \pi_{i-1})$  to obtain  $\pi_i$  and obtain the output  $y_n$  along with a succinct proof  $\pi_n$ . See e.g. [44] for more details and references.

This approach seems like the ideal tool to prove a blind rotation. The overhead for the prover of proving the verifier circuit for each iteration is relatively small compared to our function  $F$  implementing a step of the blind rotation, due to plonky2's focus on optimization of recursion. There is one caveat, in that in our description above there is no public input beyond the initial  $x$ . In particular, the individual loop iterations do not receive any public input specific to the iteration. In contrast, in our application of blind rotation, every loop iteration receives a different part of the bootstrapping key and ciphertext element. One way to solve this is by passing the entire bootstrapping key as input to the step function and use a counter that keeps track of the loop iteration. Then we could use a selector subcircuit that picks out the correct part of the key and the LWE mask for the current iteration. Note that this subcircuit grows linearly with the size of the bootstrapping


 Figure 3: Illustration of recursion-based IVC.  $F$  is the circuit computing the loop iteration,  $V$  is the verifier circuit verifying a proof for  $F'$ , the illustrated circuit itself. The public input is  $x_0$  along with the public outputs  $x_{i+1}$  and the verifier output, the other two inputs are prover inputs.

key, which consists of  $n(k+1)^2 \ell N$  elements in  $\mathbb{Z}_q$ . For small  $n$  this circuit is smaller than the circuit for our step function and thus does not incur too large of an overhead, but as  $n$  grows it quickly becomes the bottleneck.

So we opt for another solution based on hashing: since plonky2 is optimized for recursion and its verifier needs to perform hashing operations, it necessarily supports efficient proofs for evaluating hash functions. Accordingly, we let the elements of the bootstrapping key and LWE ciphertext be private prover inputs, which may differ across iterations, and extend the circuit computing the loop to compute a running hash chain over them. The final hash is part of the output and the verifier may recompute the hash in order to verify that the prover used the correct bootstrapping key and LWE ciphertext in the correct order. In fact, we split the hash over the bootstrapping key and the ciphertext into two separate hash chains. This has the advantage that for a fixed bootstrapping key the verifier needs to compute the corresponding hash chain only once, e.g. during key generation. Since the bootstrapping key is orders of magnitude larger than the ciphertext, this significantly speeds up the verifier in case multiple PBS operations per bootstrapping key are to be evaluated. Note that the verifier does not even need to store the bootstrapping key after computing this hash and may perform verification with the hash only. We remark that we do not claim novelty for the idea of replacing a large public input with a large private input and a small public hash value. This seems to be folklore in the zkVM literature and even plonky2 already employs this technique itself. The novelty here is in the observation that

it provides an elegant solution to our problem of different, and potentially very large, inputs to each loop iteration.

## 4 Extension to Full PBS

We now outline how we extend the IVC-based prover to a full PBS. In contrast to a regular, non-recursive prover, this is not trivial and we cannot simply plug together the step circuits and obtain a prover for the complete functionality. However, we will see that we can still extend the prover efficiently to the full PBS. The resulting IVC circuit is illustrated in Figure 4.

### 4.1 Mod Switch

Recall that we need to switch the modulus of the input ciphertext  $c = (a, b) \in \mathbb{Z}_q^{n+1}$  to turn it into a ciphertext  $c' = (a', b') \in \mathbb{Z}_{2N}^{n+1}$ . The resulting elements of  $c'$  are used as input to the negacyclic rotation operation (cf. Section 3.1), where they are binary decomposed and the individual bits are used as control bits of MUX operations that pick the shifted or unshifted polynomial, where the shift is fixed. It follows that an easy way to perform the mod switch is to consider the element  $a_i$  in each iteration (or  $-b$  in the first iteration), perform a bit decomposition and use the  $\log N + 1$  most significant bits as input to the polynomial rotation. In fact, in order to round to the closest integer, we use the  $\log N + 2$  most significant bits and the final shift by one position is performed twice, once with the  $(\log N + 1)$ st most significant bit and again with the  $(\log N + 2)$ nd bit (the latter leading to the correct rounding). While this approach does not perform the mod switch exactly as described in Section 2.2, it is a close enough approximation as we quantify next.

For an element  $a \in \mathbb{Z}_q$ , the mod switch operation would require to perform the operation  $a \mapsto \lfloor a \cdot 2N/q \rfloor$ . The circuit we describe above instead performs the operation  $a \mapsto \lfloor a \cdot 2N/2^{64} \rfloor$ .

**LEMMA 4.1.** *Let  $c = (a, b) \in \mathbb{Z}_q^{n+1}$  be an LWE ciphertext with binary key  $s \in \{0, 1\}^n$  and let  $p \in \mathbb{Z}$  such that  $q/p = (1 - \epsilon)$ . Then performing the mod switch to  $2N$  using  $p$  instead of  $q$  increases the error by at most  $\epsilon \cdot 2N$ .*

**PROOF.** Let  $\epsilon_b = \frac{b \cdot 2N}{p} - \left\lfloor \frac{b \cdot 2N}{p} \right\rfloor$  and  $\epsilon_i = \frac{a_i \cdot 2N}{p} - \left\lfloor \frac{a_i \cdot 2N}{p} \right\rfloor$ . Then we have

$$\begin{aligned} \left\lfloor \frac{b \cdot 2N}{p} \right\rfloor - \sum_i \left\lfloor \frac{a_i \cdot 2N}{p} \right\rfloor s_i &= \frac{2N}{p} (b - \langle a, s \rangle) + \epsilon_b - \sum_i \epsilon_i s_i \\ &= \frac{q}{p} \cdot \frac{2N}{q} (b - \langle a, s \rangle) + \epsilon_b - \sum_i \epsilon_i s_i \\ &= \frac{2N}{q} (b - \langle a, s \rangle) - \epsilon \frac{2N}{q} (b - \langle a, s \rangle) \\ &\quad + \epsilon_b - \sum_i \epsilon_i s_i \end{aligned}$$

The lemma follows, since  $(b - \langle a, s \rangle)/q < 1$  and the rounding errors  $\epsilon_b$  and  $\epsilon_i$  have a similar distribution as they would when mod switching using  $q$ .  $\square$

Since the mod switch operation itself incurs an error of  $O(\sqrt{n})$  (cf. the  $\epsilon_b - \sum_i \epsilon_i s_i$  term), it is clear that for typical parameters the additional error of  $\epsilon \cdot 2N$  is negligible if  $\epsilon \ll 1/N$ .

### 4.2 Key Switch

The biggest challenge in extending the blind rotation to a full PBS is the key switch as it is structurally quite different from the blind rotation. There are essentially two options to add the key switch in a straight-forward manner. First, one could extend the circuit to perform the full key switch in each round on the accumulator value in parallel to the blind rotation step and select the output value depending on the loop counter using a MUX. The drawback of this solution is that the circuit for a full key switch is quite large compared to a step of the blind rotation and thus would slow down each step significantly.

The second approach would be to perform just one of the  $k \cdot N + 1$  steps of the key switch in every iteration and again select the output depending on the loop counter. The overhead in each iteration would be very small and thus each iteration would be just as fast to prove as without the key switch. However, we now need to perform  $n + k \cdot N$  steps of the loop iteration instead of just  $n$ . Since  $k \cdot N$  is typically larger than  $n$ , this incurs a slowdown of at least a factor 2.

Clearly, one could attempt to mitigate above issues by implementing a hybrid, but we chose a different path. Inspired by [9] we do not perform sample extraction and then an LWE key switch, but rather first perform a GLWE key switch to a partial key of size  $n$  and then perform a trivial sample extraction on the verifier side (cf. Section 4.3). The advantage is that the GLWE key switch has the same structure as the external product but with a key switching key instead of a GGSW encryption as input. This means, we can re-use the largest part of the blind rotation circuit, the external product, for the key switch. The additional logic of selecting the input and output to the external product circuit is small in comparison and does not affect prover time, and this increases the overall number of loop iterations only by one. The drawback is that the key switch needs to use the same parameters (decomposition base and level, ring and GLWE dimension) as the blind rotation, but the key switching key needs to carry larger noise for security due to the key being partial. So this requires tweaking the parameters. Looking ahead, we note that we use the parameter optimization approach from [9] but restricting the search space such that the bootstrapping and the key switch use the same parameters.

### 4.3 Sample Extraction

Sample extraction takes as input a GLWE ciphertext and outputs an LWE ciphertext of dimension  $n = kN$  where the key of the resulting ciphertext is the (concatenation of the) coefficient vector(s) of the GLWE secret key. This conversion consists of a simple, fixed re-ordering and negation of a few elements and is thus very cheap and easy to perform. Hence, we may assume that the verifier performs it itself, i.e. we may assume the prover sends the GLWE ciphertext resulting from the PBS and GLWE key switch to the verifier and the verifier will perform the sample extraction itself. In the following we note that we can trivialize the sample extraction even further by modifying the key switching key, ensuring that the LWE sample is obtained by literally copying a subset of the coefficients from the GLWE sample.

In the following we assume  $k = 1$  for simplicity, but the generalization to  $k > 1$  is straight-forward. Let  $s' \in \mathbb{Z}^n$  be the target LWE key, i.e. the key under which the output ciphertext of the PBS

should be encrypted. (Typically, this is the same key under which the input ciphertext to the PBS is encrypted.) Let  $s \in R_q$  be the key of the GLWE ciphertext that is the result of the key switch. For a GLWE ciphertext  $(a, b)$ , write  $\tilde{m} = b - a \cdot s$ . Then we have

$$\tilde{m}_0 = b_0 - (a \cdot s)_0 = b_0 - \left( \left( \sum_i a_i \cdot X^i \right) \cdot s \right)_0 = b_0 - \sum_i a_i \cdot (X^i \cdot s)_0$$

So if we set  $s$  such that  $(X^i \cdot s)_0 = s'_i$ , we see that the coefficient vector of  $a$  together with  $b_0$  forms a valid LWE ciphertext encrypting  $\tilde{m}_0$  under  $s'$ . So by modifying the key switching key to switch to  $s$  as defined above, we may think of this modification as integrating the usual sample extraction into the key switch. This also works for  $n < N$ , since we can view  $s \in \{0, 1\}^N$  as an  $N$ -dimensional vector, where the last  $N - n$  elements are 0. This also means we can drop the corresponding elements of the extracted mask  $a$ .

## 5 Experimental Results

In this section, we describe our experimental results. As we have observed, building a circuit for the PBS is a tedious task. As such, an easier way to approach proving the correct execution of a PBS would be to implement it in a zkVM. While easy to use, zkVMs introduce a significant overhead and designing a circuit for a SNARK is typically much more efficient, especially in terms of proving time. To quantify how much exactly we are gaining from the latter approach, we first give some results on our implementation in existing zkVMs and then proceed to experimental results of our design in plonky2.

### 5.1 Zero Knowledge Virtual Machines

Modern general purpose zkVMs are computing platforms based on STARKs [7] and the RISC-V instruction set architecture. To use such zkVMs, one must write the program whose execution they would like to prove in a general purpose programming language such as Rust, which supports RISC-V as a compilation target. The compiled program is then given as input to the zkVM, which executes it and produces a proof of correct execution. Notice the subtle difference between this approach and more common verifiable computation techniques: the circuit for which the proof is generated is not that of the compiled program, but that of the virtual machine which receives the compiled program as *input*. The versatility of zkVMs makes them a powerful tool in that they allow users unfamiliar with arithmetic circuit generation or domain specific languages [4, 5, 41] to easily generate proofs for any program they have already built. However, the overhead caused by the VM logic is significant and as such, there is a trade-off between ease of use and performance.

To set a performance baseline, we use general purpose zkVMs RISC Zero [14] and SP1 [42] to generate proofs of correct execution for a step of TFHE’s bootstrapping. Since we are not able to prove a complete PBS in one go due to performance limitations, we extrapolate from micro-benchmarks to estimate real performance.

Using RISC Zero, we are able to measure that one step of the blind rotation takes about 26 million RISC-V instructions to complete. The time taken to generate a proof of this computation is about 6 minutes on average on an AWS Hpc7a.96xlarge machine. In contrast, using the SP1 zkVM, we measure that one step of the blind rotation takes about 27 million instructions. We measure a

|           | Prover time (min) | Verifier time (s) |
|-----------|-------------------|-------------------|
| RISC Zero | 6.4               | 0.5               |
| SP1       | 2.1               | 1.1               |

**Table 1: Performance of proving and verifying a single step of the blind rotation using zkVMs on an AWS Hpc7a.96xlarge machine.**

| $q$                   | $n$ | $N$      | $k$ | $B$   | $\ell$ |
|-----------------------|-----|----------|-----|-------|--------|
| $2^{64} - 2^{32} + 1$ | 728 | $2^{10}$ | 1   | $2^5$ | 4      |

**Table 2: TFHE parameters suitable for our circuit. The noise parameters are set such that we may claim 128 bits of security relying on the lattice estimator. This parameterization allows for a plaintext space of size 4.**

timing of approximately 2 minutes per step of the blind rotation on average with the same type of machine.

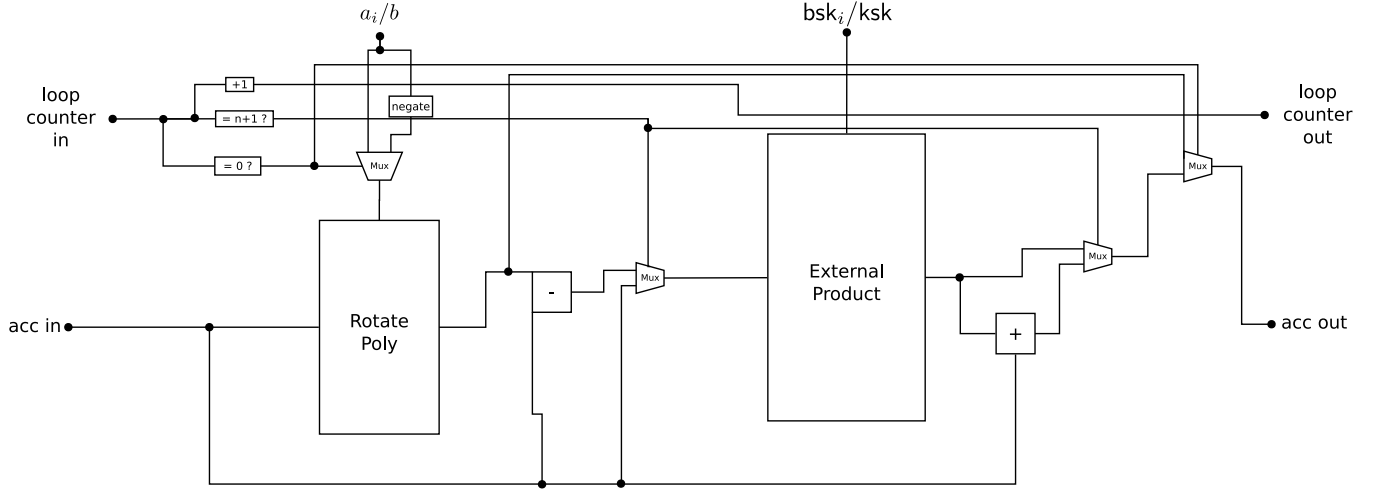
A performance comparison between the two zkVMs is presented in Table 1. Note that the timings included in this table represent a single step of the blind rotation (cf. Section 2.2.2). The time taken to prove a full PBS using a zkVM increases by a factor approximately equal to the parameter  $n$  (cf. Table 2). This is explained by proof composition and recursion techniques allowing the proving time to grow only linearly in the size of the circuit even though the underlying SNARK may not offer a linear time prover.

### 5.2 Our plonky2 based Implementation

*Parameters.* There are a multitude of parameters of TFHE’s PBS that may be tweaked and optimized, all of which impact the correctness, security and performance of the PBS. This optimization is very complex. Indeed, it is a subject of scientific research in its own right [8]. As pointed out in Section 4.2, we need to tweak parameters to ensure correctness and security. For this, we follow the approach of [9] (which in turn uses an adaptation of [8]), since some of the proposed algorithms are similar to our circuit design. In order to obtain a usable and performant set of parameters, we tweaked the optimization code from [9] by restricting the search space to fit our needs (cf. Section 4.2). We show the corresponding parameters in Table 2. We remark though that the optimization targets a computational model that is different from the arithmetic circuit model, and, as we saw in Section 3.1, some of the operations have significantly different cost in different computational models. It follows that the parameters we obtained might not minimize the circuit for the PBS and might not be optimal. However, fully optimizing parameters for our setting is out of scope of this work and we believe our results already demonstrate the progress of our approach towards practical verifiable FHE.

*Results.* We experimented with our plonky2-based implementation on a few different machines: a modern consumer laptop (M3 MacBook Pro) and a two AWS EC2 compute-optimized instances. Proof size is obviously independent of the machine and was a little less than 200 kB. In our tests with a non-recursive approach (cf.





**Figure 4: IVC Circuit for TFHE's PBS.** This corresponds to the subcircuit F from Figure 3. We omit the hash chains over the bootstrapping key and ciphertext  $(a, b)$ . The mod switch is not depicted since we consider it integrated into the polynomial rotation as described in Section 4.1.

Section 3.2) even AWS instances with large amounts of memory struggled to prove even a small-ish number of steps ( $n \approx 50$ ) due to the memory requirement. As expected, this was not the case in our experiments with the recursive IVC approach, where memory consumption is independent of the number of loop iterations  $n$ . In fact, even an older laptop with just 8 GB of memory was able to run the prover, albeit taking significantly longer than the better performing machines we report timings for in Table 3.

## 6 Security in the UC Model

The security guarantees obtained from vFHE are poorly understood, as highlighted in [38, 47]. The goal of this section is to rigorously prove a strong security notion achieved by combining a passively secure FHE scheme and a SNARK. We do so by capturing a classic use case as an MPC application and prove its security in the UC model [17]. For this, we need to first give formal definitions of the primitives involved.

### 6.1 Preliminaries

*Definition 6.1.* Let  $\mathcal{P}$ ,  $\mathcal{C}$  and  $\mathcal{F}$  be the plaintext space, ciphertext space and a circuit family, respectively. A symmetric FHE scheme  $\mathcal{E}$  for  $\mathcal{F}$  is a tuple of algorithms:

- $\mathcal{E}.\text{Gen}(1^\lambda)$ : generates a secret key  $s$  and an evaluation key  $p$
- $\mathcal{E}.\text{Enc}(s, m)$ : takes a key  $s$  and message  $m \in \mathcal{P}$  and outputs a ciphertext  $c \in \mathcal{C}$
- $\mathcal{E}.\text{Dec}(s, c)$ : takes a secret key and a ciphertext  $c \in \mathcal{C}$  and returns a message in  $\mathcal{P}$ .
- $\mathcal{E}.\text{Eval}(p, f, (c_1, \dots, c_\ell))$ : takes an evaluation key  $p$ , a circuit  $f$  in  $\mathcal{F}$  and a tuple of input ciphertexts  $(c_1, \dots, c_\ell) \in \mathcal{C}^\ell$  and returns a ciphertext  $C$ .

In the following we will assume that  $\mathcal{E}.\text{Dec}$  and  $\mathcal{E}.\text{Eval}$  are deterministic. We extend encryption and decryption to vectors of

messages and ciphertexts, resp., in the natural way, i.e. element-wise.

An FHE scheme  $\mathcal{E}$  is *correct* if for all  $(m_1, \dots, m_\ell) \in \mathcal{P}^\ell$  and all  $f \in \mathcal{F}$  we have

$$\Pr \left[ m \neq f(m_1, \dots, m_\ell) \mid \begin{array}{l} (s, p) \leftarrow \mathcal{E}.\text{Gen}(1^\lambda) \\ (c_1, \dots, c_\ell) \leftarrow \mathcal{E}.\text{Enc}(s, m_1, \dots, m_\ell) \\ c \leftarrow \mathcal{E}.\text{Eval}(p, f, c_1, \dots, c_\ell) \\ m \leftarrow \mathcal{E}.\text{Dec}(s, c, f, c_1, \dots, c_\ell) \end{array} \right]$$

is negligible in  $\lambda$ .

*Definition 6.2.* A symmetric FHE scheme  $\mathcal{E}$  is IND-CPA secure if for all PPT adversaries  $\mathcal{A}$  it holds that

$$\left| 2 \cdot \Pr \left[ b = b' \mid \begin{array}{l} (s, p) \leftarrow \mathcal{E}.\text{Gen}(1^\lambda) \\ b \leftarrow U(\{0, 1\}) \\ b' \leftarrow \mathcal{A}^{O_{Enc}^b}(p) \end{array} \right] - 1 \right| = \text{negl}(\lambda)$$

where  $O_{Enc}^b(m_0, m_1)$  is the oracle that returns  $\mathcal{E}.\text{Enc}(s, m_b)$ .

In this work we are interested in SNARKs with pre-processing. The following definition is adapted from [12, 20].

*Definition 6.3.* A preprocessing succinct non-interactive argument of knowledge (ppSNARK)  $\Pi = (\text{Gen}, \text{Prove}, \text{Verify})$  for a circuit family  $\mathcal{F}$  over some domain  $\mathcal{D}$  is a triple of algorithms such that:

- (1) Gen takes as input a security parameter  $\lambda \in \mathbb{Z}$  and circuit  $f \in \mathcal{F} : \mathcal{D}^{\ell+w} \mapsto \mathcal{D}^k$  and outputs a reference string  $\sigma$  and verification state  $\tau$ .
- (2) Prove takes as input  $\sigma$ , a statement  $(x, y) \in \mathcal{D}^{\ell+k}$  and a witness  $\omega \in \mathcal{D}^w$  and outputs a proof  $\pi$  when  $f(x, \omega) = y$ .
- (3) Verify takes as input  $\tau$ , a statement  $(x, y)$  and a proof  $\pi$  and outputs Acc or Rej.

For succinctness, we require the size of  $\pi$  and the running time of Verify to be sublinear in the size of  $f$  and  $\omega$ .

|                | CPU Cores | Memory (GB) | Prover time (min) | Verifier time (ms) |
|----------------|-----------|-------------|-------------------|--------------------|
| M3 MacBook Pro | 8         | 16          | 40                | 4                  |
| C6i.metal      | 128       | 256         | 21                | 8                  |
| Hpc7a.96xlarge | 192       | 768         | 18                | 8                  |

**Table 3: Performance of proving and verifying a PBS operation using our plonky2 based implementation. Proving benefits more from parallelization than verifying, which explains why a consumer laptop may outperform a multi-CPU machine for verification only.**

A ppSNARK is *complete* if for all  $f \in \mathcal{F}$ ,  $(x, \omega, y) \in \mathcal{D}^{\ell+w+k}$  such that  $f(x, \omega) = y$ ,

$$\Pr \left[ \text{Verify}(\tau, (x, y), \pi) = \text{Acc} \mid \begin{array}{l} (\sigma, \tau) \leftarrow \text{Gen}(\lambda, f) \\ \pi \leftarrow \text{Prove}(\sigma, (x, y), \omega) \end{array} \right] = 1$$

A ppSNARK is *knowledge sound* if for all PPT adversaries  $\mathcal{A}$  there exists an extractor  $\mathcal{E}$  such that

$$\Pr \left[ \begin{array}{l} \text{Verify}(\tau, (x, y), \pi) = \text{Acc} \\ \wedge f(x, \omega) \neq y \end{array} \mid \begin{array}{l} (\sigma, \tau) \leftarrow \text{Gen}(\lambda, f) \\ ((x, y), \pi) \leftarrow \mathcal{A}(\sigma) \\ \omega \leftarrow \mathcal{E}(\sigma) \end{array} \right] = \text{negl}(\lambda)$$

for all  $f \in \mathcal{F}$ .

## 6.2 UC Security of FHE + SNARK

Recall that in our IVC based scheme we use a hash function applied to the input to reduce the circuit size and the circuit additionally proves that the hash was computed correctly. We now show that this is sound in the UC model. A key observation here is that we are able to prove security without requiring the SNARK to be straightline extractable or blackbox. This is because in our proof the simulator does not need to run the extractor. The extractor is only used to show that the simulation is indistinguishable under the collision resistance of the hash function.

*Setting.* We will model the basic outsourcing application as a secure 2-party computation. So let  $\mathcal{I}$  be the party holding  $f, m_1, \dots, m_\ell$  that wants to obtain  $f(m_1, \dots, m_\ell)$ , where the  $m_1, \dots, m_\ell$  are private and  $f$  is public. Let  $\mathcal{A}$  be the computing party. We define the ideal functionality we are targeting below (see Algorithm 1). With every query we let  $\mathcal{I}$  specify whether or not  $m = f(m_1, \dots, m_\ell)$  is secret or public by sending a public bit  $b$ . If  $b = 0$ , then only  $\mathcal{I}$  should receive the output  $m$ , otherwise  $m$  is also sent to  $\mathcal{A}$ . In the language of encryption schemes, this models decryption oracles and thus provides security against active attacks.

We will realize the ideal functionality using a symmetric key FHE (with plaintext  $\mathcal{P}$ , ciphertext  $\mathcal{C}$  and supported circuit family  $\mathcal{F}'$ ), a collision resistant hash function  $h$ , and a suitable ppSNARK. Specifically, for any  $f \in \mathcal{F} = \{\mathcal{E}.\text{Eval}(\cdot, f', \dots) \mid f' \in \mathcal{F}'\}$  define  $f^h(w) = (f(w), h(w))$ . Then the ppSNARK we consider needs to support the circuit family  $\{f^h \mid f \in \mathcal{F}\}$ . Note that we treat  $w$  as witness, so the input  $x$  is empty (i.e.  $\ell = 0$ ). We assume that  $\mathcal{I}$  is honest, but  $\mathcal{A}$  may be fully corrupted. The protocol is the simplest one one can think of, see Algorithm 2 and Algorithm 3.

**THEOREM 6.4.** *The protocol given in Algorithm 2 and Algorithm 3 realizes the functionality from Algorithm 1 in the UC model.*

### Algorithm 1: Ideal Functionality

```

1 receive  $(f, m_1, \dots, m_\ell, b) \in \mathcal{F} \times \mathcal{P}^\ell \times \{0, 1\}$  from  $\mathcal{I}$ 
2 receive  $a \in \{0, 1\}$  from  $\mathcal{S}$ 
3 if  $a = 1$  then
4   |  $m \leftarrow \perp$ 
5 else
6   |  $m \leftarrow f(m_1, \dots, m_\ell)$ 
7 if  $b = 0$  then
8   | return  $m$  to  $\mathcal{I}$ 
9 else
10  | return  $m$  to  $\mathcal{I}$  and  $\mathcal{S}$ 
```

### Algorithm 2: Setup

```

1  $\mathcal{I}$  computes  $(s, p) \leftarrow \mathcal{E}.\text{Gen}(1^\lambda)$ 
2  $\mathcal{I}$  sends the evaluation key  $p$  to  $\mathcal{A}$ 
```

### Algorithm 3: Main Protocol

```

1  $\mathcal{I}$  receives inputs  $(f, m_1, \dots, m_\ell, b)$ 
2  $\mathcal{I}$  computes  $(c_1, \dots, c_\ell) \leftarrow \mathcal{E}.\text{Enc}(s, m_1, \dots, m_\ell)$ 
3  $\mathcal{I}$  computes  $z = h(p, c_1, \dots, c_\ell)$ 
4  $\mathcal{I}$  sends  $(f, c_1, \dots, c_\ell)$  to  $\mathcal{A}$ 
5  $\mathcal{A}$  sets  $f^h = (\mathcal{E}.\text{Eval}(\cdot, f, \cdot, \dots, \cdot), h(\cdot, \dots, \cdot))$ 
6  $\mathcal{A}$  computes  $(\sigma, \tau) \leftarrow \Pi.\text{Gen}(1^\lambda, f^h)$ ,
    $(c, z') \leftarrow f^h(p, c_1, \dots, c_\ell)$  and
    $\pi \leftarrow \Pi.\text{Prove}(\sigma, (\perp, (c, z')), (p, c_1, \dots, c_\ell))$ 
7  $\mathcal{A}$  sends  $((c, z'), \pi)$  to  $\mathcal{I}$ 
8  $\mathcal{I}$  sets  $f^h = (\mathcal{E}.\text{Eval}(\cdot, f, \cdot, \dots, \cdot), h(\cdot, \cdot, \dots, \cdot))$ 
9  $\mathcal{I}$  computes  $(\sigma, \tau) \leftarrow \Pi.\text{Gen}(1^\lambda, f^h)$  and
    $d \leftarrow \Pi.\text{Verify}(\tau, (\perp, (c, z')), \pi)$ 
10 if  $d = \text{Rej}$  or  $z \neq z'$  then
11  |  $\mathcal{I}$  sets  $m \leftarrow \perp$ 
12 else
13  |  $\mathcal{I}$  computes  $m \leftarrow \mathcal{E}.\text{Dec}(s, c)$ 
14 if  $b = 1$  then
15  |  $\mathcal{I}$  sends  $m$  to  $\mathcal{A}$ 
16  |  $\mathcal{A}$  outputs  $m$ 
17  $\mathcal{I}$  outputs  $m$ 
```

**PROOF.** We first give the simulator  $\mathcal{S}$  that has access to the ideal functionality (but not the secret input of  $\mathcal{I}$ ) in Algorithm 4. The setup in Algorithm 2 is simulated by  $\mathcal{S}$  simply by running it itself.

**Algorithm 4:** Simulator S (Main Protocol)

---

```

1 receive  $\mathcal{I}$ 's public inputs  $f$  and  $b$ 
2  $(m_1, \dots, m_\ell) \leftarrow 0^\ell$ 
3  $(c_1, \dots, c_\ell) \leftarrow \mathcal{E}. \text{Enc}(s, m_1, \dots, m_\ell)$ 
4 compute  $z = h(p, c_1, \dots, c_\ell)$  send  $(f, c_1, \dots, c_\ell)$  to  $\mathcal{A}$ 
5 receive  $(c, \pi)$  from  $\mathcal{A}$ 
6 set  $f^h = (\mathcal{E}. \text{Eval}(\cdot, f, \cdot, \dots, \cdot), h(\cdot, \cdot, \dots, \cdot))$ 
7 compute  $(\sigma, \tau) \leftarrow \Pi. \text{Gen}(1^\lambda, f^h)$  and
    $d \leftarrow \Pi. \text{Verify}(\tau, (\perp, (c, z')), \pi)$ 
8 if  $d = \text{Rej}$  or  $z \neq z'$  then
9    $a \leftarrow 1$ 
10 else
11    $a \leftarrow 0$ 
12 send  $a$  as  $\mathcal{A}$ 's input to the functionality
13 if  $b = 1$  then
14   receive the output  $m$  from the functionality and forward
     it to  $\mathcal{A}$ 

```

---

It remains to show that the simulation is indeed indistinguishable to the environment  $\mathcal{Z}$  from an execution of the real protocol. Recall that  $\mathcal{Z}$  controls the adversary  $\mathcal{A}$  and the inputs of  $\mathcal{I}$  and gets to see the output of  $\mathcal{I}$ .

The simulation differs from the execution of the real protocol in two places: 1) it uses dummy messages  $m_1, \dots, m_\ell = 0^\ell$ , and 2) it ignores the ciphertext sent by  $\mathcal{A}$  and uses the output of the ideal functionality instead. We define a modified protocol that proceeds as in the real protocol, but mirrors the ideal functionality instead of decrypting the ciphertext from  $\mathcal{A}$  (but uses the real input messages), see Algorithm 5.

**Algorithm 5:** Modified Protocol

---

```

1  $\mathcal{I}$  receives inputs  $(f, m_1, \dots, m_\ell, b)$ 
2  $\mathcal{I}$  computes  $(c_1, \dots, c_\ell) \leftarrow \mathcal{E}. \text{Enc}(s, m_1, \dots, m_\ell)$ 
3  $\mathcal{I}$  computes  $z = h(p, c_1, \dots, c_\ell)$ 
4  $\mathcal{I}$  sends  $(f, c_1, \dots, c_\ell)$  to  $\mathcal{A}$ 
5  $\mathcal{A}$  sets  $f^h = (\mathcal{E}. \text{Eval}(\cdot, f, \cdot, \dots, \cdot), h(\cdot, \cdot, \dots, \cdot))$ 
6  $\mathcal{A}$  computes  $(\sigma, \tau) \leftarrow \Pi. \text{Gen}(1^\lambda, f^h)$ ,
    $(c, z') \leftarrow f^h(p, c_1, \dots, c_\ell)$  and
    $\pi \leftarrow \Pi. \text{Prove}(\sigma, (\perp, (c, z')), (p, c_1, \dots, c_\ell))$ 
7  $\mathcal{A}$  sends  $((c, z'), \pi)$  to  $\mathcal{I}$ 
8  $\mathcal{I}$  sets  $f^h = (\mathcal{E}. \text{Eval}(\cdot, f, \cdot, \dots, \cdot), h(\cdot, \cdot, \dots, \cdot))$ 
9  $\mathcal{I}$  computes  $(\sigma, \tau) \leftarrow \Pi. \text{Gen}(1^\lambda, f^h)$  and
    $d \leftarrow \Pi. \text{Verify}(\tau, (\perp, (c, z')), \pi)$ 
10 if  $d = \text{Rej}$  or  $z \neq z'$  then
11    $\mathcal{I}$  sets  $m \leftarrow \perp$ 
12 else
13    $\mathcal{I}$  computes  $m \leftarrow f(m_1, \dots, m_\ell)$ 
14 if  $b = 1$  then
15    $\mathcal{I}$  sends  $m$  to  $\mathcal{A}$ 
16    $\mathcal{A}$  outputs  $m$ 
17  $\mathcal{I}$  outputs  $m$ 

```

---

From the view of  $\mathcal{Z}$ , the real and the modified protocol (Algorithm 5) are identical up to the point where the output  $m$  is computed in case  $d \neq \text{Rej}$ . In order to reach that part, we must have  $d = \text{Acc}$  and  $z = z'$ , so the proof sent by  $\mathcal{A}$  must verify and the hash value of the input must match. In order to distinguish the two worlds, we must now have  $\mathcal{E}. \text{Dec}(s, c) \neq f(m_1, \dots, m_\ell)$ . First, apply the extractor to the prover to obtain the witness  $w = (p', c'_1, \dots, c'_\ell)$ . By the soundness of  $\Pi$  we have  $h(w) = z'$  and since  $z' = z = h(p, c_1, \dots, c_\ell)$ , we conclude that  $p' = p$  and  $(c'_1, \dots, c'_\ell) = (c_1, \dots, c_\ell)$ , since otherwise the extractor can be used to break the collision resistance of  $h$ . Then two cases remain: either we have  $\mathcal{E}. \text{Eval}(p, f, c_1, \dots, c_\ell) \neq c$ , which means that  $\mathcal{A}$  can be used to break the soundness of  $\Pi$ ; or we have  $\mathcal{E}. \text{Eval}(p, f, c_1, \dots, c_\ell) = c$ , meaning that  $\mathcal{E}. \text{Dec}(s, c) \neq f(m_1, \dots, m_\ell)$ . This violates the correctness of  $\mathcal{E}$  and happens only with negligible probability. Note that the correctness conditions requires this to be the case for any input messages  $(m_1, \dots, m_\ell) \in \mathcal{P}^\ell$  and any  $f \in \mathcal{F}$ , so this is independent of  $\mathcal{Z}$ 's choice of  $\mathcal{I}$ 's input.

The only difference between a simulation of the real protocol (i.e. the ideal world) and of the modified protocol in  $\mathcal{Z}$ 's view is that the ciphertexts  $c_1, \dots, c_\ell$  encrypt the actual input messages in the modified protocol, while in the ideal world they encrypt  $0^\ell$ . Note that in both cases the output message is computed from the actual inputs. Clearly, the indistinguishability of the modified protocol and the ideal world follows from the IND-CPA security of  $\mathcal{E}$ .  $\square$

One may wonder, if our proposed protocol is useful from an efficiency standpoint. This is reasonable since, after all, the user  $\mathcal{I}$  needs to run  $\Pi. \text{Gen}(\cdot)$  for each query in order to obtain  $\tau$  and ppSNARGs give no guarantee that this is more efficient than simply performing the computation itself. However, from the protocol, it is clear that  $\Pi. \text{Gen}(\cdot)$  only needs to be run once per distinct  $f \in \mathcal{F}$ , so there is enormous potential for amortization over many queries to the same function. Furthermore, note that the preprocessing as we defined it is public and transparent. This means that the computing party  $\mathcal{A}$  could perform the preprocessing, send  $\tau$  to the user  $\mathcal{I}$  and commit to it by, e.g. uploading a hash of  $(f, \tau)$  to a blockchain. Now, any party, including competitors in a market-based application, can check the validity of the setup. If the user  $\mathcal{I}$  is willing to rely on such non-cryptographic incentives to ensure the integrity of the verification state  $\tau$ , it never needs to perform  $\Pi. \text{Gen}(\cdot)$ . In fact,  $\mathcal{I}$  never even has to read the entire circuit  $f$  in this setting.

## 7 Applications

### Privacy-preserving blockchains

As mentioned in section Section 1.1, techniques presented in this paper can be used to alleviate the computational load of blockchain validators participating in FHE-based privacy-preserving blockchain protocols. Consider for example a blockchain rollup [45] with a dispute resolution period of 7 days. Suppose the rollup is deployed in a way such that proofs of correct blockchain state transition are only required when a dispute occurs (i.e., it is an hybrid rollup). Arguably, at current performance levels, our prototype can already be used in such applications: since the party responsible of providing a proof of correct blockchain state transition has one week to do so, it has more than enough time to generate a proof involving FHE

computation. Further optimizing our prototype and combining it with hardware acceleration will reduce proving time and enable proof generation of more complex computations, indicating that our approach is a first step in the right direction towards practical verifiable FHE.

## Privacy-preserving machine learning

In what follows we slightly augment our prototype to demonstrate how it can enable novel applications in privacy-preserving machine learning.

Privacy-preserving machine learning applications such as [36, 49] rely on FHE to compute common ML operations (e.g., weighted sums and activation functions) over encrypted user data. Such frameworks allow for a two-party protocol where a client sends a private input (e.g., an encrypted prompt) to a server, which evaluates an inference function on this prompt and returns the encrypted result to the client. However, clients in this scenario can suffer from attacks if the server deviates from the prescribed protocol [23]. Hence we show how our prototype can be used to strengthen security in this use case by generating a proof of correct inference for a rudimentary feedforward neural network.

**Weighted sum.** The weighted sum operation is at the center of contemporary machine learning models. Therefore, the prototype is modified to compute a weighted sum of the form  $\sum_{i=1}^n w_i \cdot x_i$  where the weights  $w_i$  are in cleartext and the user inputs  $x_i$  are ciphertexts. The prover and verifier algorithms must now take in a vector of ciphertexts  $\mathbf{c} = (c_1, \dots, c_n)$  as input. The weights are assumed to be constants known in advance<sup>6</sup> and built in the arithmetic circuit. This assumption is realistic because model weights are fixed after training and do not change from one inference to another. Now remember from Section 2.2.2 that the blind rotation step circuit takes in as input a single element  $a$  of the ciphertext being bootstrapped and uses this element to compute two outputs: 1) the output of the MUX operation, which involves switching the modulus of  $a$  and performing a negacyclic rotation, and 2) a hash chain over these ciphertext elements. These two functionalities must be amended as follows for the prover to be able to generate proofs of correct weighted sum computation.

For the first part, the step circuit is modified to receive as input a vector of ciphertext elements  $\mathbf{a} = (a_{c_1}, \dots, a_{c_n})$  instead of a single ciphertext element. At blind rotation step  $j$ , this vector is comprised of all  $j$ -th elements of the ciphertexts in  $\mathbf{c}$ . Before the modulus switch step, the circuit now multiplies each element  $a_{c_i}$  with its respective weight  $w_i$  before summing them together, resulting in a combined element  $\hat{a} = \sum_{i=1}^n w_i \cdot a_{c_i}$ . Note that given two LWE ciphertexts  $c = (a_1, \dots, a_n, b)$ ,  $c' = (a'_1, \dots, a'_n, b')$  encrypting  $m$  and  $m'$  respectively and a scalar  $w \in \mathbb{Z}_p$ , we have that  $w \cdot c = (w \cdot a_1, \dots, w \cdot a_n, w \cdot b)$  and  $c + c' = (a_1 + a'_1, \dots, a_n + a'_n, b + b')$  which decrypt to  $w \cdot m$  and  $m + m'$  respectively provided ciphertext noise keeps small. As such, the element  $\hat{a}$  computed by the circuit is equivalent to the  $j$ -th element of the ciphertext  $\hat{c} = \sum_{i=1}^n w_i \cdot c_i$ . This ciphertext element  $\hat{a}$  is then used as is for the modulus switch procedure and the negacyclic rotation, meaning that the step circuit is now effectively performing a step of the blind rotation for the

ciphertext  $\hat{c}$ . Therefore, the ciphertext output by the circuit is the result of a bootstrapped weighted sum of ciphertexts.

Second, the circuit now computes one hash chain per input ciphertext. This implies that the input and output of the step circuit increase in size linearly with the number of input ciphertexts  $n$ , but has little impact on the prover and verifier complexity and does not affect the security guarantees discussed in Section 3.2.1.

**Activation function.** The original prototype takes in as input an LWE ciphertext  $c$  encrypting  $m$  and encodes the identity function  $\text{id}(m) = m$  in the test polynomial used in the bootstrapping. As such, both the input and output ciphertexts of the bootstrapping encrypt the same message and the operation only serves to reduce the noise. However, as mentioned in Section 2.2.2, it is possible to exploit the structure of the bootstrapping to evaluate a univariate function  $f$  on  $m$  by encoding  $f$  into the test polynomial. For example, the test polynomial can be set to encode the commonly used rectified linear unit (ReLU) activation function in order to obtain an encryption of  $\text{ReLU}(m)$  “for free” as a result of bootstrapping  $c$ . By combining this with the weighted sum modification, one can now generate a proof of correct execution of a rudimentary feedforward neural network of the form  $\text{ReLU}(\sum_{i=1}^n w_i \cdot x_i)$  where the  $x_i$  are encrypted and potentially sensitive user inputs.

With 4 inputs, this combination only marginally increases the circuit size as well as prover and verifier complexity. We leave as future work finding better optimized TFHE parameters that would allow for more extensive testing of this application. This amendment to our prototype demonstrates how it can be used to construct verifiable private machine learning inference and thus strengthen security guarantees in the typical two-party computation protocol proven secure above.

## Acknowledgements

We would like to thank Samuel Tap for help with the parameter optimization and the RISC Zero team for assistance with their zkVM.

## References

- [1] Adi Akavia, Craig Gentry, Shai Halevi, and Margarita Vald. 2022. Achievable CCA2 Relaxation for Homomorphic Encryption. In *TCC 2022, Part II (LNCS, Vol. 13748)*, Eike Kiltz and Vinod Vaikuntanathan (Eds.). Springer, Cham, 70–99. doi:10.1007/978-3-031-22365-5\_3
- [2] Martin R. Albrecht, Rachel Player, and Sam Scott. 2015. On the concrete hardness of Learning with Errors. *J. Math. Cryptol.* 9, 3 (2015), 169–203.
- [3] Diego F. Aranha, Anamaria Costache, Antonio Guimarães, and Eduardo Soria-Vazquez. 2024. HELIOPOLIS: Verifiable Computation over Homomorphically Encrypted Data from Interactive Oracle Proofs is Practical. In *ASIACRYPT 2024, Part V (LNCS, Vol. 15488)*, Kai-Min Chung and Yu Sasaki (Eds.). Springer, Singapore, 302–334. doi:10.1007/978-981-96-0935-2\_10
- [4] Aztec. 2023. The Noir Programming Language. <https://noir-lang.org/>. Accessed: 2024-03-01.
- [5] Marta Bellés-Muñoz, Miguel Isabel, Jose Luis Muñoz-Tapia, Albert Rubio, and Jordi Baylina Melé. 2023. Circom: A Circuit Description Language for Building Zero-Knowledge Applications. *IEEE Trans. Dependable Secur. Comput.* 20, 6 (2023), 4733–4751. doi:10.1109/TDSC.2022.3232813
- [6] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. 2018. Fast Reed-Solomon Interactive Oracle Proofs of Proximity. In *ICALP 2018 (LIPIcs, Vol. 107)*, Ioannis Chatzigiannakis, Christos Kaklamanis, Daniel Marx, and Donald Sannella (Eds.). Schloss Dagstuhl, 14:1–14:17. doi:10.4230/LIPIcs.ICALP.2018.14
- [7] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. 2018. Scalable, transparent, and post-quantum secure computational integrity. *Cryptology ePrint Archive*, Paper 2018/046. <https://eprint.iacr.org/2018/046> <https://eprint.iacr.org/2018/046>

<sup>6</sup>Specifically, the weights are agreed upon by the prover and verifier during the transparent setup phase of the SNARK protocol.

- [8] Loris Bergerat, Anas Boudi, Quentin Bourgerie, Ilaria Chillotti, Damien Ligier, Jean-Baptiste Orfila, and Samuel Tap. 2023. Parameter Optimization and Larger Precision for (T)FHE. *Journal of Cryptology* 36, 3 (July 2023), 28. doi:10.1007/s00145-023-09463-5
- [9] Loris Bergerat, Ilaria Chillotti, Damien Ligier, Jean-Baptiste Orfila, Adeline Roux-Langlois, and Samuel Tap. 2024. New Secret Keys for Enhanced Performance in (T)FHE. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS 2024, Salt Lake City, UT, USA, October 14-18, 2024*, Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie (Eds.). ACM, 2547–2561. doi:10.1145/3658644.3670376
- [10] Alexandre Bois, Ignacio Cascudo, Dario Fiore, and Dongwoo Kim. 2021. Flexible and Efficient Verifiable Computation on Encrypted Data. In *PKC 2021, Part II (LNCS, Vol. 12711)*, Juan Garay (Ed.). Springer, Cham, 528–558. doi:10.1007/978-3-030-75248-4\_19
- [11] Dan Boneh and Binyi Chen. 2024. LatticeFold: A Lattice-based Folding Scheme and its Applications to Succinct Proof Systems. Cryptology ePrint Archive, Paper 2024/257. <https://eprint.iacr.org/2024/257>
- [12] Dan Boneh, Yuval Ishai, Amit Sahai, and David J. Wu. 2017. Lattice-Based SNARGs and Their Application to More Efficient Obfuscation. In *EUROCRYPT 2017, Part III (LNCS, Vol. 10212)*, Jean-Sébastien Coron and Jesper Buus Nielsen (Eds.). Springer, Cham, 247–277. doi:10.1007/978-3-319-56617-7\_9
- [13] Sean Bowe, Jack Grigg, and Daira Hopwood. 2019. Halo: Recursive Proof Composition without a Trusted Setup. Cryptology ePrint Archive, Report 2019/1021. <https://eprint.iacr.org/2019/1021>
- [14] Jeremy Bruestle, Paul Gafni, and RISC Zero. 2023. RISC Zero zkVM: Scalable, Transparent Arguments of RISC-V Integrity. <https://dev.risczero.com/proof-system-in-detail.pdf>. Accessed: 2024-02-29.
- [15] Chris Brzuska, Sébastien Canard, Caroline Fontaine, Duong Hieu Phan, David Pointcheval, Marc Renard, and Renaud Sirdey. 2025. Relations Among New CCA Security Notions for Approximate FHE. *IACR Communications in Cryptology (CIC)* 2, 1 (2025), 20. doi:10.62056/aee0iv7sf
- [16] Benedikt Bünz and Binyi Chen. 2023. Protostar: Generic Efficient Accumulation/Folding for Special-Sound Protocols. In *ASIACRYPT 2023, Part II (LNCS, Vol. 14439)*, Jian Guo and Ron Steinfeld (Eds.). Springer, Singapore, 77–110. doi:10.1007/978-981-99-8724-5\_3
- [17] Ran Canetti. 2001. Universally Composable Security: A New Paradigm for Cryptographic Protocols. In *42nd FOCS*. IEEE Computer Society Press, 136–145. doi:10.1109/SFCS.2001.959888
- [18] Sylvain Chatel, Christian Knabenhans, Apostolos Pyrgelis, and Jean-Pierre Hubaux. 2022. Verifiable Encodings for Secure Homomorphic Analytics. *CoRR* abs/2207.14071 (2022).
- [19] Sylvain Chatel, Christian Knabenhans, Apostolos Pyrgelis, Carmela Troncoso, and Jean-Pierre Hubaux. 2023. Poster: Verifiable Encodings for Maliciously-Secure Homomorphic Encryption Evaluation. In *CCS*. ACM, 3525–3527.
- [20] Alessandro Chiesa. 2014. *Succinct non-interactive arguments*. Ph. D. Dissertation. Massachusetts Institute of Technology, Cambridge, MA, USA.
- [21] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. 2016. Faster Fully Homomorphic Encryption: Bootstrapping in Less Than 0.1 Seconds. In *ASIACRYPT 2016, Part I (LNCS, Vol. 10031)*, Jung Hee Cheon and Tsuyoshi Takagi (Eds.). Springer, Berlin, Heidelberg, 3–33. doi:10.1007/978-3-662-53887-6\_1
- [22] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. 2020. TFHE: Fast Fully Homomorphic Encryption Over the Torus. *Journal of Cryptology* 33, 1 (Jan. 2020), 34–91. doi:10.1007/s00145-019-09319-x
- [23] Ilaria Chillotti, Nicolas Gama, and Louis Goubin. 2016. Attacking FHE-based applications by software fault injections. Cryptology ePrint Archive, Report 2016/1164. <https://eprint.iacr.org/2016/1164>
- [24] Ilaria Chillotti, Damien Ligier, Jean-Baptiste Orfila, and Samuel Tap. 2021. Improved Programmable Bootstrapping with Larger Precision and Efficient Arithmetic Circuits for TFHE. In *ASIACRYPT 2021, Part III (LNCS, Vol. 13092)*, Mehdi Tibouchi and Huaxiong Wang (Eds.). Springer, Cham, 670–699. doi:10.1007/978-3-030-92078-4\_23
- [25] Morten Dahl, Clément Danjou, Daniel Demmler, Tore Frederiksen, Petar Ivanov, Marc Joye, Dragos Rotaru, Nigel Smart, and Louis Tremblay Thibault. 2023. fhEVM: Confidential EVM Smart Contracts using Fully Homomorphic Encryption. <https://github.com/zama-ai/fhevm/blob/main/fhevm-whitepaper.pdf>. Accessed: 2023-11-22.
- [26] Dario Fiore, Rosario Gennaro, and Valerio Pasto. 2014. Efficiently Verifiable Computation on Encrypted Data. In *ACM CCS 2014*, Gail-Joon Ahn, Moti Yung, and Ninghui Li (Eds.). ACM Press, 844–855. doi:10.1145/2660267.2660366
- [27] Dario Fiore, Anca Nitulescu, and David Pointcheval. 2020. Boosting Verifiable Computation on Encrypted Data. In *PKC 2020, Part II (LNCS, Vol. 12111)*, Aggelos Kiayias, Markulf Kohlweiss, Petros Wallden, and Vassilis Zikas (Eds.). Springer, Cham, 124–154. doi:10.1007/978-3-030-45388-6\_5
- [28] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. 2019. PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge. Cryptology ePrint Archive, Report 2019/953. <https://eprint.iacr.org/2019/953>
- [29] Chaya Ganes, Anca Nitulescu, and Eduardo Soria-Vazquez. 2023. Rinocchio: SNARKs for Ring Arithmetic. *Journal of Cryptology* 36, 4 (Oct. 2023), 41. doi:10.1007/s00145-023-09481-3
- [30] Sanjam Garg, Aarushi Goel, and Mingyuan Wang. 2024. How to Prove Statements Obviously?. In *CRYPTO 2024, Part X (LNCS, Vol. 14929)*, Leonid Reyzin and Douglas Stebila (Eds.). Springer, Cham, 449–487. doi:10.1007/978-3-031-68403-6\_14
- [31] Rosario Gennaro, Craig Gentry, and Bryan Parno. 2010. Non-interactive Verifiable Computing: Outsourcing Computation to Untrusted Workers. In *CRYPTO 2010 (LNCS, Vol. 6223)*, Tal Rabin (Ed.). Springer, Berlin, Heidelberg, 465–482. doi:10.1007/978-3-642-14623-7\_25
- [32] Craig Gentry, Amit Sahai, and Brent Waters. 2013. Homomorphic Encryption from Learning with Errors: Conceptually-Simpler, Asymptotically-Faster, Attribute-Based. In *CRYPTO 2013, Part I (LNCS, Vol. 8042)*, Ran Canetti and Juan A. Garay (Eds.). Springer, Berlin, Heidelberg, 75–92. doi:10.1007/978-3-642-40041-4\_5
- [33] Shafi Goldwasser, Yael Tauman Kalai, Raluca A. Popa, Vinod Vaikuntanathan, and Nikolai Zeldovich. 2013. How to Run Turing Machines on Encrypted Data. In *CRYPTO 2013, Part II (LNCS, Vol. 8043)*, Ran Canetti and Juan A. Garay (Eds.). Springer, Berlin, Heidelberg, 536–553. doi:10.1007/978-3-642-40084-1\_30
- [34] Marc Joye. 2022. SoK: Fully Homomorphic Encryption over the [Discretized] Torus. *IACR TCHES* 2022, 4 (2022), 661–692. doi:10.46586/tches.v2022.i4.661-692
- [35] Abhiram Kothapalli, Srinath Setty, and Ioanna Tziavala. 2022. Nova: Recursive Zero-Knowledge Arguments from Folding Schemes. In *CRYPTO 2022, Part IV (LNCS, Vol. 13510)*, Yevgeniy Dodis and Thomas Shrimpton (Eds.). Springer, Cham, 359–388. doi:10.1007/978-3-031-15985-5\_13
- [36] Joon-Woo Lee, HyungChul Kang, Yongwoo Lee, Woosuk Choi, Jieun Eom, Maxim Deryabin, Eunsang Lee, Junghyun Lee, Donghoon Yoo, Young-Sik Kim, and Jong-Seon No. 2022. Privacy-Preserving Machine Learning With Fully Homomorphic Encryption for Deep Neural Network. *IEEE Access* 10 (2022), 30039–30054. doi:10.1109/ACCESS.2022.3159694
- [37] Fengrun Liu, Haofei Liang, Tianyu Zhang, Yuncong Hu, Xiang Xie, Haisheng Tan, and Yu Yu. 2025. HasteBoots: Proving FHE Bootstrapping in Seconds. Cryptology ePrint Archive, Paper 2025/261. <https://eprint.iacr.org/2025/261>
- [38] Mark Manulis and Jérôme Nguyen. 2024. Fully Homomorphic Encryption Beyond IND-CCA1 Security: Integrity Through Verifiability. In *EUROCRYPT 2024, Part II (LNCS, Vol. 14652)*, Marc Joye and Gregor Leander (Eds.). Springer, Cham, 63–93. doi:10.1007/978-3-031-58723-8\_3
- [39] Polygon. 2022. Plonky2. <https://github.com/mir-protocol/plonky2>. Accessed: 2023-11-22.
- [40] Nigel P. Smart. 2023. Practical and Efficient FHE-Based MPC. In *19th IMA International Conference on Cryptography and Coding (LNCS, Vol. 14421)*, Elizabeth A. Quaglia (Ed.). Springer, Cham, 263–283. doi:10.1007/978-3-031-47818-5\_14
- [41] Starknet. 2020. The Cairo Programming Language. <https://www.cairo-lang.org/>. Accessed: 2024-03-01.
- [42] Succinct. 2024. SP1. <https://github.com/succinctlabs/sp1/>. Accessed: 2024-02-29.
- [43] The Fhenix Team. 2023. FHE-Rollups: Scaling Confidential Smart Contracts on Ethereum and Beyond. [https://www.fhenix.io/wp-content/uploads/2023/11/FHE\\_Rollups\\_Whitepaper-v0.1-1.pdf](https://www.fhenix.io/wp-content/uploads/2023/11/FHE_Rollups_Whitepaper-v0.1-1.pdf). Accessed: 2023-11-22.
- [44] Justin Thaler. 2022. Proofs, Arguments, and Zero-Knowledge. *Found. Trends Priv. Secur.* 4, 2-4 (2022), 117–660.
- [45] Louis Tremblay Thibault, Tom Saray, and Abdelhakim Senhaji Hafid. 2022. Blockchain Scaling Using Rollups: A Comprehensive Survey. *IEEE Access* 10 (2022), 93039–93054. doi:10.1109/ACCESS.2022.3200051
- [46] Paul Valiant. 2008. Incrementally Verifiable Computation or Proofs of Knowledge Imply Time/Space Efficiency. In *TCC 2008 (LNCS, Vol. 4948)*, Ran Canetti (Ed.). Springer, Berlin, Heidelberg, 1–18. doi:10.1007/978-3-540-78524-8\_1
- [47] Alexander Viand, Christian Knabenhans, and Anwar Hithnawi. 2023. Verifiable Fully Homomorphic Encryption. arXiv:2301.07041 [cs.CR]
- [48] Michael Walter. 2024. What Have SNARGs Ever Done for FHE? Cryptology ePrint Archive, Report 2024/1207. <https://eprint.iacr.org/2024/1207>
- [49] Zama. 2022. Concrete ML: a Privacy-Preserving Machine Learning Library using Fully Homomorphic Encryption for Data Scientists. <https://github.com/zama-ai/concrete-ml>
- [50] Zama. 2022. TFHE-rs: A Pure Rust Implementation of the TFHE Scheme for Boolean and Integer Arithmetics Over Encrypted Data. <https://github.com/zama-ai/tfhe-rs>